

Predictive Action Projection with PPO

Software and Artifact Companion: Operational Mental Models and Repository Workflow

Salvador Tenorio

Software and artifact companion

Abstract

This companion explains how to reason operationally about the predictive action-projection repository. It complements the formal implementation design by concentrating on the mental models needed to read the code, trace one action through the system, understand Gymnasium and PyTorch boundaries, interpret projection and action-bound diagnostics, audit CSV and NPZ evidence, and distinguish tests, validation, calibration, experiments, and paper artifacts. The intended reader may have substantial software-development experience but limited prior exposure to reinforcement-learning research code. The repository is independent research software: its purpose is transparent numerical experimentation and the development of research judgment, not production-platform generality. Exact mathematical and persisted-data contracts remain authoritative in the implementation design, focused Markdown contracts, and source code.

Scope boundary

This companion explains implemented components, artifact meanings, and expected repository workflows. It is not a validation report, calibration record, experiment-status update, or presentation of empirical results. Exact status and campaign decisions belong to revision-specific validation evidence, frozen protocol artifacts, research records, and the scientific report.

Contents

1	Role of the companion	3
2	Research software rather than a production service	3
3	Component mental models	4
3.1	Environment as the physical simulator boundary	4
3.2	Wrappers as coordinate and behavior adapters	4
3.3	Actor-critic as two learned functions	5
3.4	Projector as a numerical supervisor	5
3.5	Evaluation as frozen-policy experimentation	5
3.6	Analysis as a pure artifact transformation	5
4	The complete action path	5
4.1	Four action representations	5
4.2	Two corrections with different meanings	6
4.3	Likelihood ownership	6
4.4	Practical debugging sequence	7

5	Arrays, tensors, shapes, and dtypes	7
5.1	NumPy–PyTorch boundary	7
5.2	Fixed observation capacity	7
5.3	Vector rollout shapes	7
5.4	Trajectory alignment	8
6	Same-step autoreset	8
7	Projection outcomes and diagnostic interpretation	9
7.1	Identity and intervention	9
7.2	Correction burden	9
7.3	Slack burden	9
7.4	Failure as unknown rather than zero	9
8	Evaluation modes and outcome transitions	10
8.1	Deterministic and stochastic policies	10
8.2	Paired evaluation	10
8.3	Outcome transitions	11
9	Evidence artifacts	11
9.1	Checkpoints	11
9.2	TensorBoard event files	11
9.3	Episode CSV files	11
9.4	Trajectory NPZ archives	12
9.5	Hashes and semantic identities	12
9.6	PASS summaries	12
10	Tests, regressions, calibration, and experiments	12
10.1	Tests	12
10.2	Regression validation	12
10.3	Pre-experiment validation	13
10.4	Calibration	13
10.5	Final experiment campaign	13
11	Aggregation and statistical units	13
12	Repository reading path	14
13	Common misconceptions	14
14	Completed-release workflow	15
15	Operational checklists	15
15.1	Before editing code	15
15.2	Before running training	15
15.3	Before accepting evaluation artifacts	16
15.4	Before interpreting results	16
16	Compact refresher	16

1 Role of the companion

The documentation set separates exact design, operational understanding, focused contracts, execution guidance, research records, campaign protocols, and empirical reporting:

- the implementation design defines the mathematical, software, artifact, and verification contracts;
- this companion explains how those contracts appear during practical code reading and experiment work;
- focused Markdown contracts define exact subsystem interfaces, schemas, metrics, and failure semantics;
- validation guidance identifies supported executable gates and evidence locations;
- records preserve regression, calibration, freeze, and provenance decisions;
- machine-readable protocols declare campaign-specific methods, seeds, modes, suites, and required evidence;
- the scientific report presents final evidence and conclusions.

This companion is not an alternative source of truth. When a field name, array shape, fallback rule, or protocol group matters, consult the implementation design, the relevant Markdown contract, and the current code.

2 Research software rather than a production service

The repository has no long-running service, request handler, domain database, or deployment topology. It consists of numerical components and explicit workflows:

$$\text{configuration} \longrightarrow \text{training or evaluation} \longrightarrow \text{saved evidence} \longrightarrow \text{audit and interpretation.} \quad (1)$$

A successful run is not defined by process uptime. It is defined by whether the command produced scientifically interpretable artifacts whose identities, parameters, and relationships can be reconstructed later.

For an experienced production engineer, the most important transition is from object-centric contracts to numerical and evidentiary contracts. In this repository, all of the following are part of an interface:

- array shape and dtype;
- coordinate system and physical units;
- seed and source revision;
- action ownership;
- terminal-event handling;
- schema version and semantic hash;
- the distinction between zero, missing, and unknown values;
- the aggregation unit used to support a claim.

A function can execute without exception and still invalidate a study by changing one of these contracts silently. The repository therefore favors inspectability, explicit artifacts, and narrow reusable workflows over a generalized experiment platform.

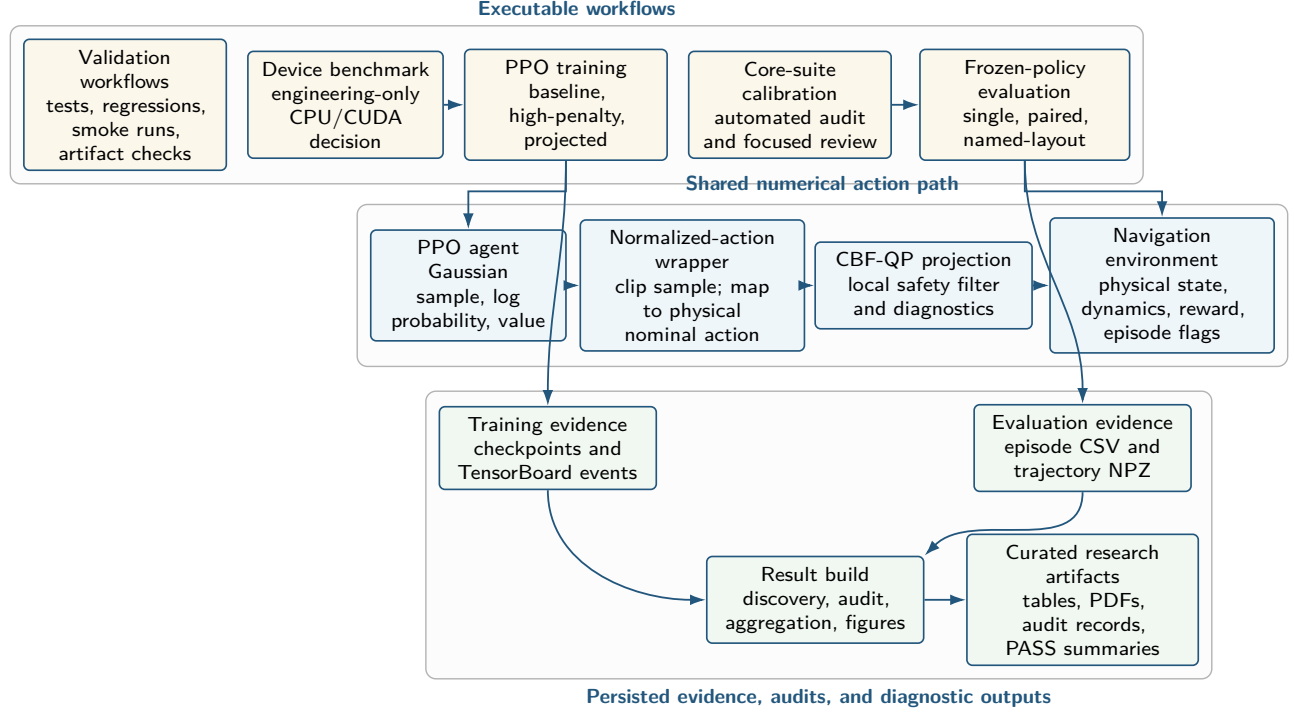


Figure 1: Numerical components, executable workflows, and persisted evidence in the repository.

3 Component mental models

3.1 Environment as the physical simulator boundary

`ConstrainedNavigationEnv` owns position, heading, goal, obstacle arrays, step count, reward, collision, success, and truncation. It is the only component that advances physical state. Its public contract is the `Gymnasium` API:

$$\text{reset}(\text{seed}, \text{options}) \mapsto (\text{observation}, \text{info}), \quad (2)$$

$$\text{step}(\text{action}) \mapsto (\text{observation}, \text{reward}, \text{terminated}, \text{truncated}, \text{info}). \quad (3)$$

The base environment accepts a physical action $[v, \omega]$. It knows nothing about PPO probabilities or normalized Gaussian samples. This separation is what allows a baseline and a projected run to share identical dynamics and reward code.

3.2 Wrappers as coordinate and behavior adapters

`Gymnasium` wrappers surround an environment and transform actions or returned data without owning the underlying physical state. The construction is

$$\begin{aligned} \text{base environment} &\longrightarrow \text{optional projection wrapper} \\ &\longrightarrow \text{normalized action wrapper} \\ &\longrightarrow \text{optional episode statistics.} \end{aligned} \quad (4)$$

Because actions move inward, the outer normalized wrapper clips and converts the policy command before the inner projection wrapper sees it. The projection therefore operates in physical units.

A wrapper-order change is comparable to changing an API's units while retaining the same method name: the program may still run, but the scientific meaning is different.

3.3 Actor–critic as two learned functions

The PPO agent follows the inspectable CleanRL-style organization and implements PPO with generalized advantage estimation [3, 5, 4]. It is one PyTorch module with two conceptual outputs:

- the actor defines a probability distribution over normalized samples;
- the critic estimates the expected return from the current observation.

The actor is not a deterministic controller even when evaluation later uses its mean. During training it samples from an unbounded diagonal Gaussian. The critic does not decide which action reaches the environment; it supplies value estimates used to compute advantages.

3.4 Projector as a numerical supervisor

The projector is not another neural network and is not trained by PPO. Its construction is related to CBF-QP safety filters [1], while the raw-versus-executed learning distinction is consistent with the projection-based analysis of Gros, Zanon, and Bemporad [2]. Given current geometry and a physical nominal command, it solves a small convex QP and returns a physical executed command plus diagnostics. It can act as the identity map, make a correction, use slack, or fail explicitly.

The filter is local. It does not search for a path around an obstacle. It may turn a collision into a timeout because preventing one immediate unsafe command does not create the long-horizon route behavior the policy failed to learn.

3.5 Evaluation as frozen-policy experimentation

Evaluation restores a checkpoint, fixes policy parameters, and runs complete episodes. It may change the evaluation seed, layout, stochastic/deterministic mode, and projection mode, but it never updates the network. This makes a same-checkpoint projection-off/on comparison interpretable: the learned parameters are fixed and only the execution filter changes.

3.6 Analysis as a pure artifact transformation

Analysis should be thought of as a compiler from validated evidence to tables and figures. It discovers artifacts, rejects incomplete or inconsistent datasets, aggregates them in the declared hierarchy, and writes scientific tables and diagnostic figures. The paper and its figure-production sources are maintained separately. It does not train or evaluate a policy.

4 The complete action path

4.1 Four action representations

The most important operational distinction in the repository is the four-stage action chain:

$$a_t^{\text{sample}} \longrightarrow a_t^{\text{bounded}} \longrightarrow u_t^{\text{raw}} \longrightarrow u_t^{\text{exec}}. \quad (5)$$

1. a_t^{sample} is the unbounded normalized Gaussian sample generated and scored by PPO.
2. $a_t^{\text{bounded}} = \text{clip}(a_t^{\text{sample}}, -1, 1)$ is the bounded normalized intermediate value.
3. u_t^{raw} is the physical nominal action produced by the affine wrapper map, denoted u_t^{nom} in the paper. Existing schema field names remain unchanged.
4. u_t^{exec} is the physical action the base environment receives after optional projection.

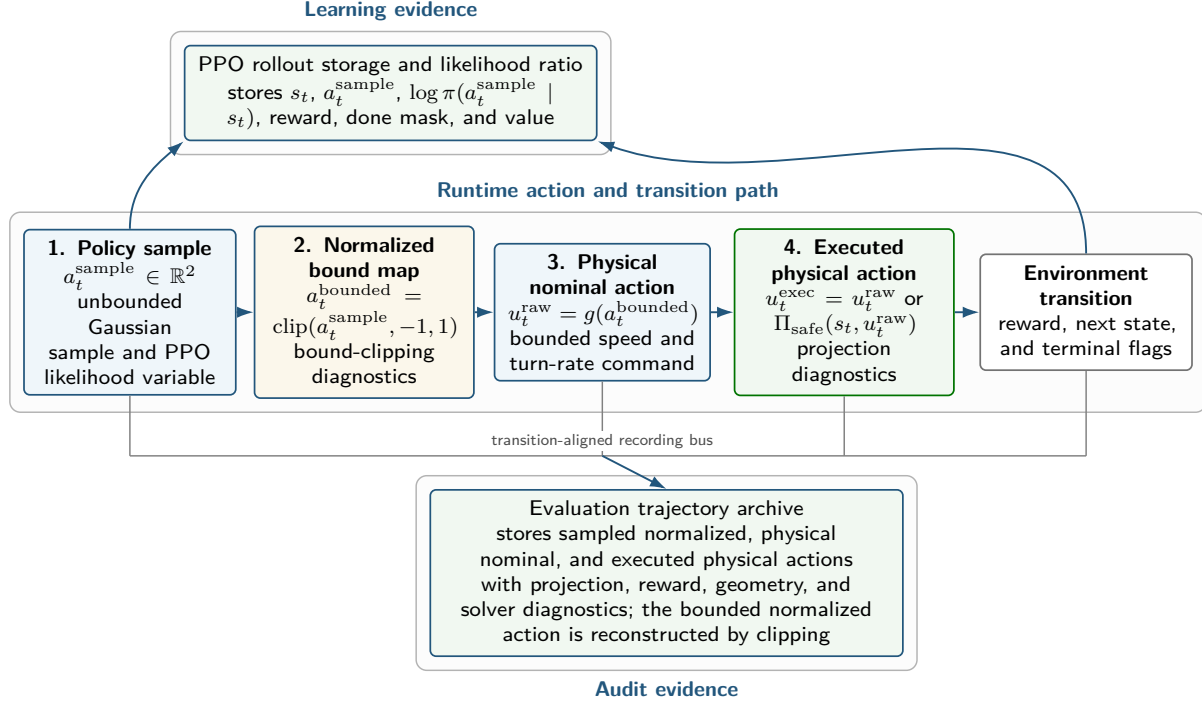


Figure 2: The exact action path. PPO owns the sample and likelihood; the environment advances under the executed physical action.

4.2 Two corrections with different meanings

Action-bound clipping measures

$$c_t^{\text{bound}} = a_t^{\text{sample}} - a_t^{\text{bounded}} \quad (6)$$

in normalized coordinates. Projection measures

$$c_t^{\text{proj}} = u_t^{\text{exec}} - u_t^{\text{raw}} \quad (7)$$

in physical speed/turn-rate coordinates.

The first reveals how often the Gaussian distribution proposes values outside the policy-facing action box. The second reveals how much the safety filter changes an already bounded physical command. Combining them would hide whether a problem originates in the actor distribution, the physical projection geometry, or both.

4.3 Likelihood ownership

PPO evaluates

$$\log \pi_{\theta}(a_t^{\text{sample}} | s_t), \quad (8)$$

because a_t^{sample} is the random variable sampled from its distribution. It does not evaluate the likelihood of a_t^{bounded} or u_t^{exec} as though those values had been sampled directly. Rewards and successor states nevertheless arise from u_t^{exec} .

This is easiest to understand by treating clipping, physical conversion, and projection as a state-dependent action transformation inside the effective environment. The implementation remains an empirical PPO formulation; it does not claim that PPO clipping inherits an exact unbiased projected-gradient theorem.

4.4 Practical debugging sequence

When an action-related result looks surprising, inspect the stages in this order:

1. the actor mean and standard deviation;
2. the sampled normalized action;
3. whether either normalized component was clipped;
4. the mapped physical nominal speed and turn rate;
5. whether projection intervened and by how much;
6. the solver status and slack;
7. the physical state transition and outcome.

This sequence prevents a collision or saturation symptom from being attributed to the wrong layer.

5 Arrays, tensors, shapes, and dtypes

5.1 NumPy–PyTorch boundary

The environment and projection code use NumPy arrays because Gymnasium and CVXPY operate naturally in that representation. The actor–critic and PPO optimizer use PyTorch tensors because gradients are required. The training loop therefore performs an explicit boundary crossing:

$$\text{NumPy observation} \rightarrow \text{PyTorch tensor} \rightarrow \text{sample tensor} \rightarrow \text{NumPy environment action}. \quad (9)$$

The sample stored in the rollout remains a tensor. The value sent through wrappers is detached and moved to CPU because the current projection is not differentiable through PyTorch.

5.2 Fixed observation capacity

The observation length is $6 + 5N_{\max}$. Obstacle capacity and active count are different concepts:

- capacity determines neural-network input size and checkpoint compatibility;
- active count determines which obstacle slots affect the current episode.

Inactive slots are zeros. Increasing active count within the existing capacity preserves the network shape; increasing capacity changes it.

5.3 Vector rollout shapes

With 256 steps and four environments, a rollout contains 1024 transitions. Typical arrays have shapes

$$\text{obs} : [256, 4, d_{\text{obs}}], \quad (10)$$

$$\text{actions_raw} : [256, 4, 2], \quad (11)$$

$$\text{logprobs, rewards, dones, values} : [256, 4]. \quad (12)$$

Before PPO optimization, the leading two dimensions are flattened. Action-bound diagnostics are computed from the complete `actions_raw` tensor and must therefore cover exactly 1024 sampled actions. Projection diagnostics must independently cover the same 1024 executed transitions, including terminal values preserved under `final_info`. A projection-count mismatch commonly indicates that terminal information was omitted or counted twice.

5.4 Trajectory alignment

An evaluated episode with T transitions has $T + 1$ states:

$$s_0, a_0, r_1, s_1, \dots, a_{T-1}, r_T, s_T. \quad (13)$$

Reward r_{t+1} belongs to the transition from s_t to s_{t+1} under the executed action. A zero-based stored reward array still has entry `rewards[t]` equal to r_{t+1} . Positions and headings therefore have one more row than actions and rewards in the NPZ archive. An off-by-one error here can make a visually plausible trajectory scientifically unusable.

6 Same-step autoreset

Under Gymnasium same-step autoreset, an environment that ends an episode returns the initial observation of the next episode in the ordinary observation array. The terminal episode’s final information is preserved separately under `final_info`.

The correct mental model is:

$$\text{one executed transition} \rightarrow \begin{cases} \text{ordinary next observation,} & \text{episode continues,} \\ \text{reset observation + terminal final_info,} & \text{episode ends.} \end{cases} \quad (14)$$

The trainer stores the transition once, uses the returned reset observation for continued vector collection, and reads terminal return/outcome/projection data from `final_info`. Projection diagnostics deliberately inspect both the ordinary mapping and `final_info`; tests and validators require complete transition coverage.

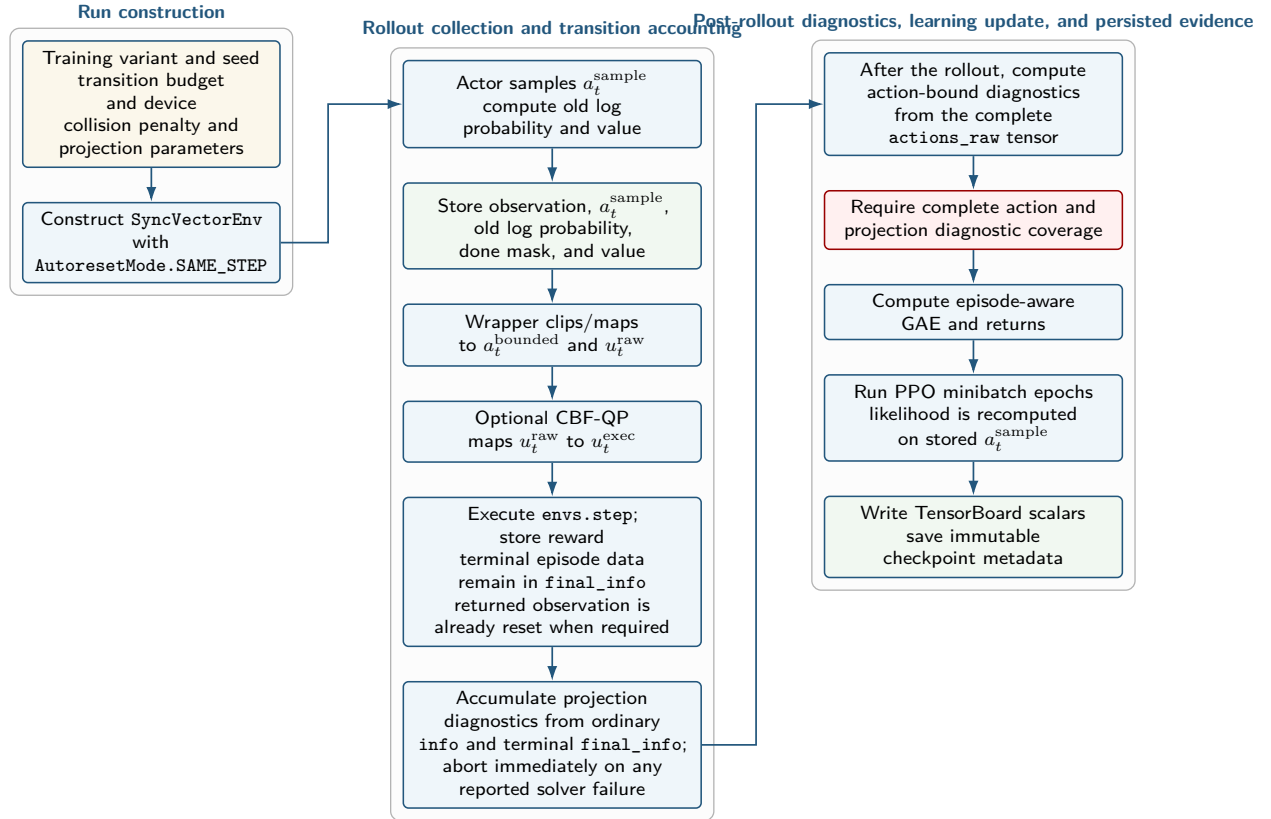


Figure 3: Rollout collection under same-step autoreset and the diagnostic coverage checks that protect terminal transitions.

7 Projection outcomes and diagnostic interpretation

7.1 Identity and intervention

Projection-enabled does not mean every action is changed. When no obstacle constraint is active, the projector returns the bounded physical nominal action and records status `no_active_constraints`. When the QP solution differs from the nominal action by more than the tolerance, `projection_intervened` is true.

Zero deterministic intervention can therefore be an important noninterference result rather than evidence that projection was accidentally disabled.

7.2 Correction burden

Mean correction norm includes zero-correction transitions. It answers: how much action modification did the complete episode or rollout experience on average? Maximum correction answers: how severe was the single largest change? Intervention rate answers: how often was any material change made? These metrics are related but not interchangeable.

7.3 Slack burden

Slack relaxes a local CBF row. Mean summed slack measures sustained aggregate relaxation across obstacles and transitions; maximum slack measures the worst individual value. Slack is not identical to collision penetration, and a small nonzero value is not automatically a solver failure.

7.4 Failure as unknown rather than zero

A successful zero-slack solve is evidence that no relaxation was used. A failed solve does not reveal how much slack would have been necessary; active slack is therefore unknown and stored as `NaN`. The stationary fallback, false success flag, and status make the failure auditable. Projected training aborts rather than allowing this event into a final learning dataset.

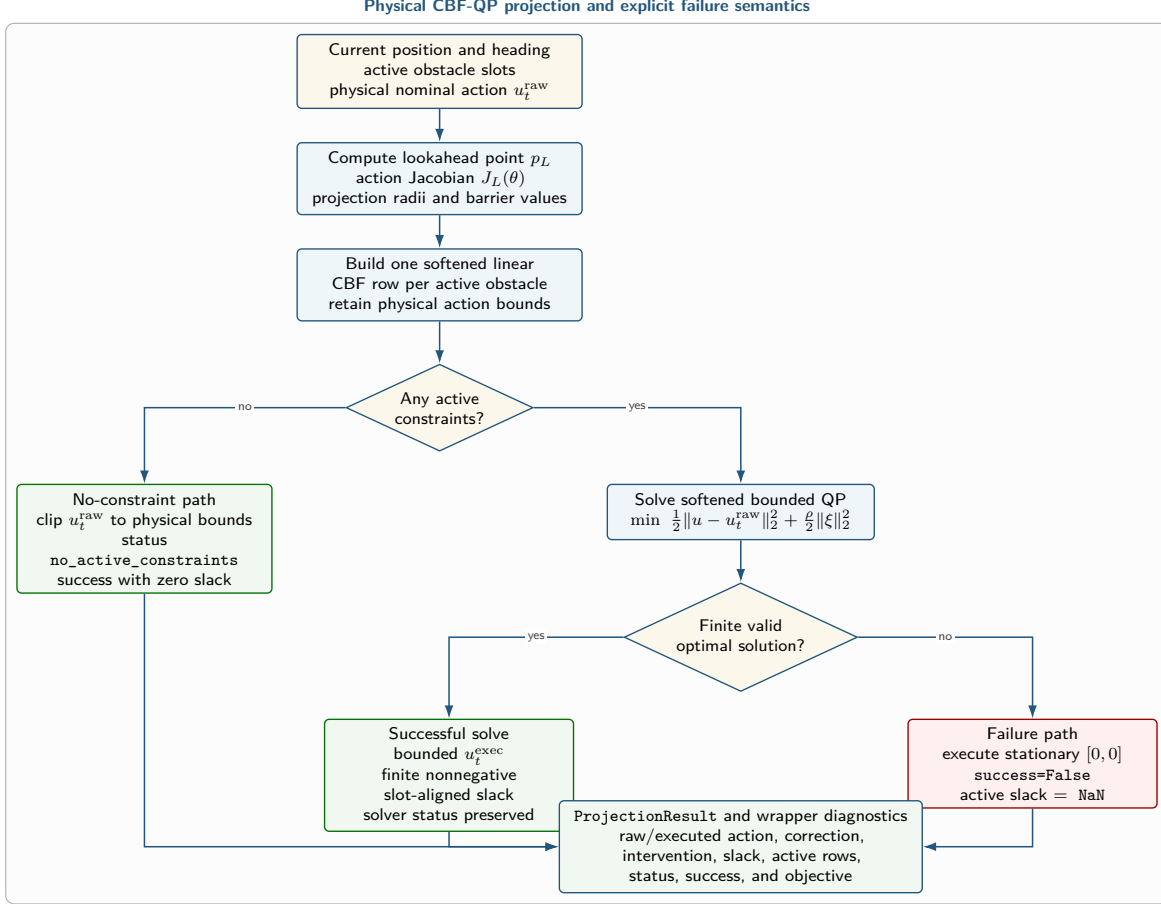


Figure 4: No-constraint, valid-solve, and failure paths of the physical action projector.

8 Evaluation modes and outcome transitions

8.1 Deterministic and stochastic policies

Deterministic actor-mean evaluation removes policy sampling and is well suited to repeatable task and layout comparisons. Stochastic evaluation samples from the learned distribution and reveals safety behavior hidden by a benign mean, including bound clipping and active projection.

These modes answer different questions. A stochastic validation run that proves intervention occurs is not automatically a final method comparison; a deterministic zero-intervention episode does not prove the full policy distribution is feasible.

8.2 Paired evaluation

Paired evaluation uses the same checkpoint, episode seeds, environment configuration, and reward while toggling projection. It is stronger than comparing unrelated runs because the learned parameters are fixed. For stochastic pairing, random state is reset so both modes begin with the same sample stream; the stream may lead to different later states after projection changes an action.

8.3 Outcome transitions

A paired row records a numeric delta and a matched terminal-outcome correspondence. The two executions can follow different states and nominal actions. The table does not establish a physical conversion or an isolated causal mechanism:

Projection off	Projection on	Interpretation
collision	success	OFF collided; ON completed the task.
collision	timeout	OFF collided; ON timed out without completion.
collision	collision	Both executions collided; delay or severity requires separate evidence.
success	success	Both executions completed; paths and correction burden may differ.
success	timeout/collision	Only OFF completed; inspect the paired trajectories for context.
timeout	success	OFF timed out; ON completed the task.
timeout	timeout	Neither execution completed; clearance and correction burden may differ.

9 Evidence artifacts

9.1 Checkpoints

A checkpoint is a model plus provenance. It records network parameters, full arguments, method label, seed, global step, shapes, dimensions, run identity, and actual device. The evaluator uses this metadata to reject incompatible environments and the paired evaluator verifies that the checkpoint hash remains unchanged.

A checkpoint designated as a regression or calibration fixture is not interchangeable with an experiment replicate merely because its transition budget matches a later campaign budget.

9.2 TensorBoard event files

Event files contain two kinds of information:

- completed-episode outcomes, which occur at irregular global steps;
- rollout-level PPO/action/projection diagnostics, which occur once per rollout iteration.

The analysis layer converts these events into aligned tables rather than plotting tag streams ad hoc. Complete protocols require the expected tags; missing diagnostic tags are an artifact failure, not a plotting inconvenience.

9.3 Episode CSV files

A CSV is the primary episode-level evidence. One row should be readable as a complete statement: which policy/checkpoint/seed/layout/mode was evaluated, what happened, what action-bound and projection burden occurred, and under which reward/projection parameters.

CSV rows are ideal for filtering, pairing, and aggregation, but they do not preserve the within-episode sequence that explains why an outcome occurred.

9.4 Trajectory NPZ archives

An NPZ archive stores named NumPy arrays. The schema contains static geometry, $T + 1$ state arrays, and T transition arrays. It preserves all three externally meaningful action representations: the unsquashed normalized sample, physical nominal action, and physical executed action.

NPZ permits a direct audit of claims such as:

- the physical nominal action equals the documented map of the clipped sample;
- the correction equals executed minus nominal physical action;
- CSV intervention and clipping rates match transition-level arrays;
- slack and failure conventions are internally consistent;
- a plotted path corresponds to the declared layout and checkpoint.

9.5 Hashes and semantic identities

Checkpoint hashes identify model bytes. Layout-suite hashes identify canonical parsed JSON content, not line endings. Protocol IDs identify expected methods and evidence groups. These different identities should not be reduced to a filename convention.

9.6 PASS summaries

A PASS summary is a transaction boundary: the validator writes it only after every required phase has completed. Partial CSV files or console output do not substitute for the marker. The detailed log explains the first failed condition.

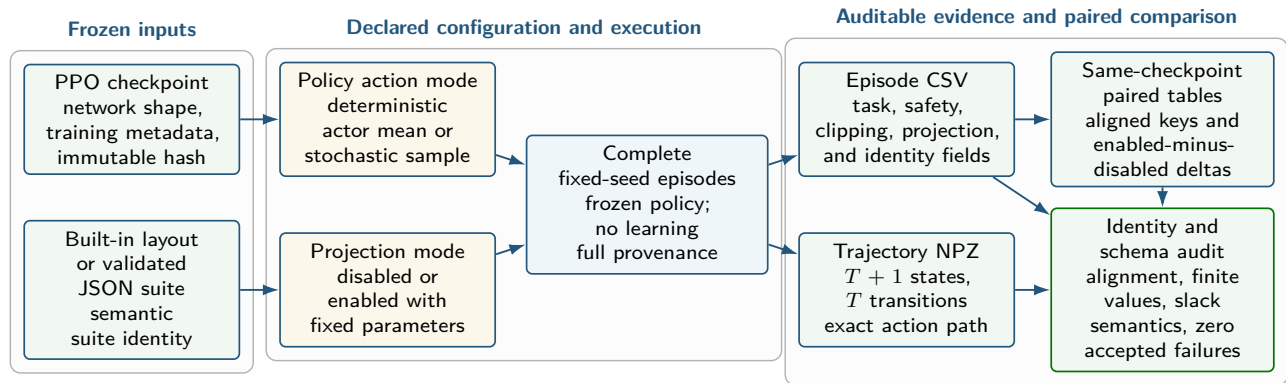


Figure 5: Frozen-policy evaluation and its episode, trajectory, pairing, and audit artifacts.

10 Tests, regressions, calibration, and experiments

10.1 Tests

Tests protect deterministic mechanics: shapes, wrapper transformations, projection rows and fallback, autoreset accounting, layout validation, archive alignment, aggregation order, and orchestration safety. They are intentionally lightweight. PPO convergence or a favorable collision result is not a unit-test expectation.

10.2 Regression validation

An evaluation-time regression uses a named checkpoint fixture and expected mechanics to detect changes in source behavior. Its purpose is not to estimate a method effect. Any PASS record remains

evidence only for the revision, environment, and fixture named by that record.

10.3 Pre-experiment validation

The consolidated pre-experiment gate is designed to perform short training for all declared methods, validate checkpoints and event tags, run development evaluation, audit CSV and NPZ action paths, build tables and figures, and provide manual-review instructions. Its role is to establish that the selected source revision and environment satisfy the declared mechanics before final runs begin.

10.4 Calibration

Calibration evaluates whether a proposed benchmark is malformed, trivial, impossible, nonintervening everywhere, constantly intervened, or numerically unstable. The historical checkpoint is a probe. The suite is not changed to improve that checkpoint’s score; only policy-independent defects justify revision.

10.5 Final experiment campaign

The completed campaign has three training histories with five independently initialized runs each, seeds 1–5, 51,200 transitions per run on CPU, and one retained final checkpoint per run. Every checkpoint was evaluated with Projector OFF and Projector ON in both frozen suites. The primary suite contains 3,000 stochastic fixed-geometry episodes; the secondary suite contains 720 actor-mean executions across 24 prespecified transfer layouts. They remain separate evidence families.

The OFF/ON conditions and transfer suite were fixed before final outcomes were observed. The decision to organize the paper around runtime attribution followed examination of the completed evidence and is explicitly retrospective. Final analysis reads saved artifacts. New experiments would require a separate versioned protocol; release verification does not repeat this campaign.

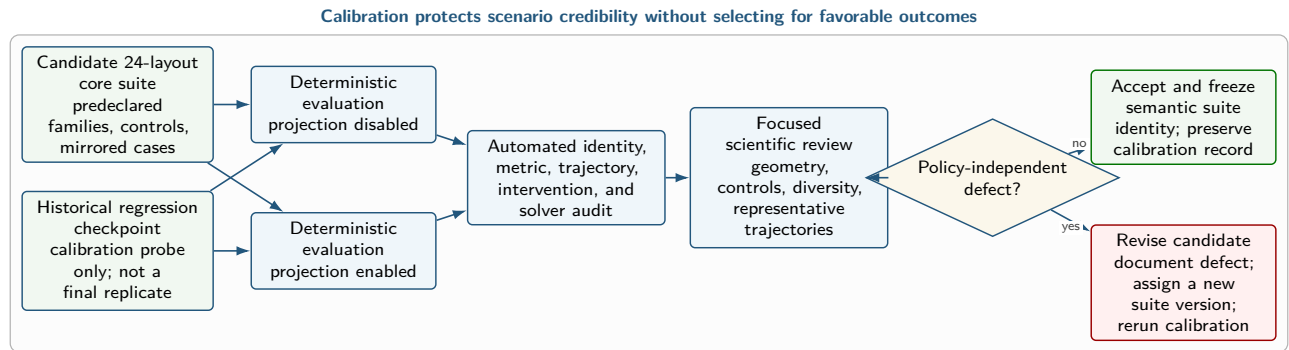


Figure 6: Calibration protects scenario credibility without selecting layouts for favorable policy outcomes.

11 Aggregation and statistical units

The empirical replicate is an independently initialized training run, represented by one retained final checkpoint. There are five runs per training history. Layouts and evaluation episodes are repeated observations of that checkpoint. The correct aggregation order is therefore

$$\begin{aligned}
 \text{episodes and layouts within checkpoint} &\longrightarrow \text{checkpoint summary} \\
 &\longrightarrow \text{method summary across training seeds.}
 \end{aligned}
 \tag{15}$$

Treating 24 layouts as 24 independently learned policies would be pseudoreplication. The analysis code protects this hierarchy and retains individual seed summaries. With five independent runs per training history, interpretation should emphasize individual runs, effect magnitude, and consistency rather than strong significance claims.

12 Repository reading path

A reader should trace one numerical path before reading orchestration code:

1. `environments/constrained_navigation.py`: state, observation, physical dynamics, reward, outcomes;
2. `environments/action_wrappers.py`: normalized clipping and physical mapping;
3. `projection/cbf_qp_projection.py`: geometry, QP, success/failure results;
4. `projection/cbf_qp_wrapper.py`: placement of projection inside the wrapper stack;
5. `algorithms/ppo/agent.py`: Gaussian actor and value critic;
6. `algorithms/ppo/action_bound_diagnostics.py` and `projection_training.py`: separate rollout diagnostics;
7. `algorithms/ppo/ppo_continuous_action.py`: complete collection/update path;
8. `evaluation/evaluate_policy.py`: one complete evaluated episode;
9. `evaluation/trajectory_recording.py`: persisted transition semantics;
10. `evaluation/evaluate_projection_pair.py` and `evaluate_layout_suite.py`: controlled comparison workflows;
11. `analysis/training_diagnostics.py` and `aggregate_projection_results.py`: evidence transformation and aggregation;
12. validation modules and records: gates, provenance, and freeze decisions.

Reading batch files first gives the wrong impression that orchestration is the algorithm. The batch files are only command aliases.

13 Common misconceptions

Misconception	Correct interpretation
The PPO action is always in $[-1, 1]^2$.	The Gaussian sample is unbounded; the wrapper clips it before physical conversion.
The raw action in every file means the same object.	The design distinguishes normalized sample, bounded normalized action, physical nominal action, and executed action. Field names must be read with their schema.
Projection-enabled means every step is changed.	The projector can return the nominal action with no active constraints or a zero correction.
Zero slack and solver failure are equivalent.	Zero slack is a valid successful result; failure stores false success and unknown active slack.
A collision penalty enforces safety.	It changes reward incentives but cannot block the current command.
Projection is a route planner.	It is a local action filter and may prevent collision without reaching the goal.

Misconception	Correct interpretation
PPO becomes SAC-like off-policy because the action is projected.	PPO remains a fresh-rollout method acting in an effective transformed environment; exact projected-gradient correctness is a separate issue.
More layouts mean more training replicates.	Layouts are tests of one checkpoint; independent training seeds are replicates.
A passing test suite proves the method works.	It proves declared mechanics; experiments determine empirical outcomes.
A PASS record proves all future revisions.	PASS evidence is revision-specific; a changed source revision requires the applicable gate to be run again.
A figure can be regenerated from memory or live code.	Diagnostic figures are generated from audited saved artifacts under a declared protocol.

14 Completed-release workflow

After applying the repository patch and copying the three PDFs compiled from the separate Overleaf projects, use the release checks with `RL_PROJECTS` active:

```
python -m pytest -q -rs
python -m evaluation.verify_repository_release --base-ref main --require-tracked
```

Stage only the repository files before using `--require-tracked`. All `runs/` artifacts remain ignored and preserved in local archives. The default verifier checks curated tables, protected source, documentation links, and staged content with no run archive present. Optional `--archive-root` verification checks preserved raw artifacts separately. Neither mode runs policies. Manuals are checked as externally compiled PDFs; their bytes may differ between compiler builds. Scientific evidence remains protected by the recorded hashes. See `docs/validation/release_verification.md` for the exact integrity boundary. The earlier pre-experiment validator is a development tool, not an additional merge gate.

15 Operational checklists

15.1 Before editing code

- Identify the mathematical or schema contract affected.
- Determine whether the change alters checkpoint compatibility, suite identity, pairing keys, or archive semantics.
- Locate the protecting tests, validators, and documentation.
- Avoid introducing a second implementation path for convenience.

15.2 Before running training

- Use the declared environment and active interpreter.
- Run the complete validation gate for the selected source revision.
- Preserve the selected device decision separately from research results.
- Verify method, seed, budget, collision penalty, projection parameters, and output path.
- Refuse accidental checkpoint overwrite.

15.3 Before accepting evaluation artifacts

- Confirm checkpoint and suite hashes.
- Confirm complete episode keys and expected modes.
- Check mutually exclusive success/collision/timeout outcomes.
- Check action-bound metrics and physical action identities against trajectories.
- Require zero accepted projection failures.
- Preserve raw CSV and NPZ evidence.

15.4 Before interpreting results

- Separate task performance, safety outcomes, bound clipping, and projection burden.
- Examine paired outcome transitions, not only mean return deltas.
- Aggregate within checkpoint before across seeds.
- Inspect representative trajectories for mechanisms and failure modes.
- State whether evidence is deterministic transfer, stochastic policy behavior on the fixed training geometry, or another explicitly declared evaluation condition.
- Keep claims within the benchmark and protocol boundary.

16 Compact refresher

Core mental model

PPO samples and scores a_t^{sample} . The normalized wrapper clips it to a_t^{bounded} and maps it to the physical nominal command u_t^{raw} . The optional CBF-QP maps that command to u_t^{exec} . The environment advances under u_t^{exec} . Bound clipping and projection correction are separate burdens. Evaluation freezes the policy; trajectories preserve the complete action path; analysis aggregates layouts within checkpoint and checkpoints across training seeds.

Core interpretation

A low collision rate is incomplete evidence. It must be read together with success, timeout, clearance, action-bound clipping, intervention rate, correction magnitude, slack, and solver status. Heavy intervention can mean the filter rather than the learned policy supplies safety.

Core evidence rule

Implementation presence, a revision-specific PASS record, a frozen campaign protocol, and an experimental result are different evidence classes. The repository records them separately so that implementation progress is not mistaken for scientific evidence.

References

- [1] Aaron D. Ames et al. “Control Barrier Function Based Quadratic Programs for Safety Critical Systems”. In: *IEEE Transactions on Automatic Control* 62.8 (2017), pp. 3861–3876. DOI: 10.1109/TAC.2016.2638961.
- [2] Sébastien Gros, Mario Zanon, and Alberto Bemporad. “Safe Reinforcement Learning via Projection on a Safe Set: How to Achieve Optimality?” In: *IFAC-PapersOnLine* 53.2 (2020), pp. 8076–8081. DOI: 10.1016/j.ifacol.2020.12.2276.

- [3] Shengyi Huang et al. “CleanRL: High-Quality Single-File Implementations of Deep Reinforcement Learning Algorithms”. In: *Journal of Machine Learning Research* 23.274 (2022), pp. 1–18. URL: <https://www.jmlr.org/papers/v23/21-1342.html>.
- [4] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations*. 2016. arXiv: 1506.02438.
- [5] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017). DOI: 10.48550/arXiv.1707.06347.